CLASS 1

1. Introduction to Full Stack Web Development

1.1 What is a Full Stack?

A stack refers to all of the components needed to run an application. This includes frameworks and components internal to an application, such as UI frameworks, MVC frameworks, data access libraries, etc. It can also include things external to an application, such as the operating system, application server, and database.

A full-stack web developer is a developer that has general knowledge across a wide breadth of technologies and platforms as well as in-depth experience and specialization in a couple of those concepts. For the most part, there are two general fields that make up a full-stack developer's skillset: front-end development and back-end development.

Front-End Development

This skillset involves the actual presentation of your website—how the information in your website is laid out in browsers and on mobile devices as well. A dedicated front-end developer will be very experienced working with HTML and CSS as well as the scripting language, JavaScript. With these languages, the developer can very efficiently manipulate the information on a website to make it appealing and effective.

Everything that you actually see on a website—the layout, the positioning of text and images, colors, fonts, buttons, and so on—are all factors that the front-end developer must consider.

The main goal of a front-end developer is to provide the platform for visitors to interact with, a platform which provides and receives information. This means some developers will be well-versed in web design and using software such as Photoshop and Illustrator to create graphics and themed layouts.

Additional skillsets of a front-end developer could include user experience design and user interface design, skills which help a team evaluate the best methods of displaying and collecting information. A front-end developer who possesses these design skills is potentially more valuable as they can identify the look and feel of a site while assessing the technical capabilities of such a design at the same time.

Back-End Development

Create, edit/update and recollection of data are some of the processes that are most often associated with back-end development. Some examples of common scripting languages used are JavaScript, PHP, Ruby, and Python. With these languages, a back-end developer can create algorithms and business logic to manipulate the data that was received in front-end development.

This means that a back-end developer must be able to write code to receive the information input from the user and also save it somewhere – like in a database.

There are two main types of databases:

- Relational (Oracle, SQL Server, PostgreSQL, MySQL)
- Non-relational (Mongo, Oracle, Couchbase)

The language used for database management is SQL, which helps the developer interact with the database.

The concepts might sound foreign, but just understand that there are different database management systems based on convenience and use.

Another component of back-end development is server management, which are applications that host the database and serve up the website. An alternative to knowing how to manage servers is to use cloud-based platforms that provide the infrastructure, like Heroku or Amazon Web Services.

Understanding server management allows a developer to troubleshoot slow applications and even determine how scalable their websites are to include more users.

Frameworks

Rather than having to develop complex proprietary code every time for creating different websites, frameworks have become popular resources to make many processes more efficient and convenient.

Libraries like jQuery are extremely popular for front-end developers using Javascript, as they can implement various functions that other developers have already cultivated and tested.

Javascript frameworks like AngularJS and EmberJS solve many of the challenges faced by front-end developers by developing conventions that can easily be implemented with any website.

On the backend, there are frameworks like Express for Node.js (JavaScript), Rails for the programming language of Ruby, Django for Python, and CakePHP for working with PHP.

The main purpose of frameworks is to make a developer's job easier by developing a set of conventions that can be adopted for many of the different processes involved in creating a website—from how information is displayed to how it is stored and accessed in the database.

1.2 The MEAN Stack?

The term MEAN stack refers to a collection of JavaScript based technologies used to develop web applications. MEAN is an acronym for

- MongoDB
- Express
- AngularJS
- Node.js

From client to server to database, MEAN is full stack JavaScript.

Node.js is a server-side JavaScript execution environment. It is a platform built on Google Chrome's V8 JavaScript runtime. It helps in building highly scalable and concurrent applications rapidly.

Express is lightweight framework used to build web applications in Node. It provides a number of robust features for building single and multi-page web applications. Express is inspired by the popular Ruby framework, Sinatra.

MongoDB is a schema-less NoSQL database system. MongoDB saves data in binary JSON format which makes it easier to pass data between client and server.

AngularJS is a JavaScript framework developed by Google. It provides some awesome features like the two-way data binding. It is a complete solution for rapid and powerful front-end development.

1.3 Installing of software components

Node.js

To install Node, you simply have to download and unzip the files into a folder of your choice. Then update the path system variable to point to the main folder. See the installation document in Canvas.

NPM:

Node Package Manager (npm) comes automatically with the node installation, however, it may be outdated. But there should be no immediate need to update it, but feel free to update npm to the latest version.

Code Editor:

Simply put, you just need a text editor. I will use Notepad++ in class, but feel free to choose any good text editor of your choice. And you may use an Integrated Development Environment (IDE), such as Atom, Visual Studio, etc.

MongoDB:

I am using a cloud, free instance of MongoDB at mongodb.com (https://www.mongodb.com/cloud/atlas). So, I recommend to create a free account at MongoDB Atlas.

You can also download MongoDB and install it locally, if you wish.

Browser:

I will be using Chrome, Firefox, and Edge in class. Again, you can use any browser of your choice, in general.

2. Basics of HTML

2.1 Overview of HTML

If you can, imagine a time before the invention of the Internet. Websites did not exist, and books, printed on paper and tightly bound, were your primary source of information. It took a considerable amount of effort—and reading—to track down the exact piece of information you were after.

Today you can open a web browser, jump over to your search engine of choice, and search away. Any bit of imaginable information rests at your fingertips. And chances are someone somewhere has built a website with your exact search in mind.

HTML, **H**yper**T**ext **M**arkup **L**anguage, gives content structure and meaning by defining that content as, for example, headings, paragraphs, or images.

CSS, or Cascading Style Sheets, is a presentation language created to style the appearance of content—using, for example, fonts or colors.

The two languages—HTML and CSS—are independent of one another and should remain that way. CSS should not be written inside of an HTML document and vice versa. As a rule, HTML will always represent content, and CSS will always represent the appearance of that content.

Understanding Common HTML Terms

While getting started with HTML, you will likely encounter new—and often strange—terms. Over time you will become more and more familiar with all of them, but the three common HTML terms you should be familiar with are:

- Elements
- Tags
- Attributes

2.2 HTML Tags

Elements

Elements are designators that define the structure and content of objects within a page. Some of the more frequently used elements include multiple levels of headings (identified as h1 through h6 elements) and paragraphs (identified as the p element); the list goes on to include the a, div, span, strong, and em elements, and many more.

Elements are identified by the use of less-than and greater-than angle brackets, < >, surrounding the element name. Thus, an element will look like the following:

Code Example:

```
<a>
```

Tags

The use of less-than and greater-than angle brackets surrounding an element creates what is known as a tag. Tags most commonly occur in pairs of opening and closing tags.

An opening tag marks the beginning of an element. It consists of a less-than sign followed by an element's name, and then ends with a greater-than sign; for example, .

A closing tag marks the end of an element. It consists of a less-than sign followed by a forward slash and the element's name, and then ends with a greater-than sign; for example, .

The content that falls between the opening and closing tags is the content of that element, if applicable. An anchor link, for example, will have an opening tag of <a> and a closing tag of . What falls between these two tags will be the content of the anchor link.

So, anchor tags will look a bit like this:

Code Example:

```
<a>…..</a>
```

Attributes

Attributes are properties used to provide additional information about an element. The most common attributes include:

- id attribute: which identifies an element
- class attribute, which classifies an element
- src attribute, which specifies a source for embeddable content
- href attribute, which provides a hyperlink reference to a linked resource

Attributes are defined within the opening tag, after an element's name. Generally, attributes include a name and a value. The format for these attributes consists of the attribute name followed by an equal sign and then a quoted attribute value.

Syntax:

< *tag_name attribute_name* **=** **"**value**"** **>**

For example, an <a> element including an href attribute would look like the following:

Code Example:

```
<a href=" https://www.google.com/">Google</a>
```

The preceding code will display the text "Google" on the web page and will take users to https://www.google.com/ upon clicking the "Google" text.

The anchor element is declared with the opening <a> and closing tags encompassing the text, and the hyperlink reference attribute and value are declared with href=" https://www.google.com/" in the opening tag.

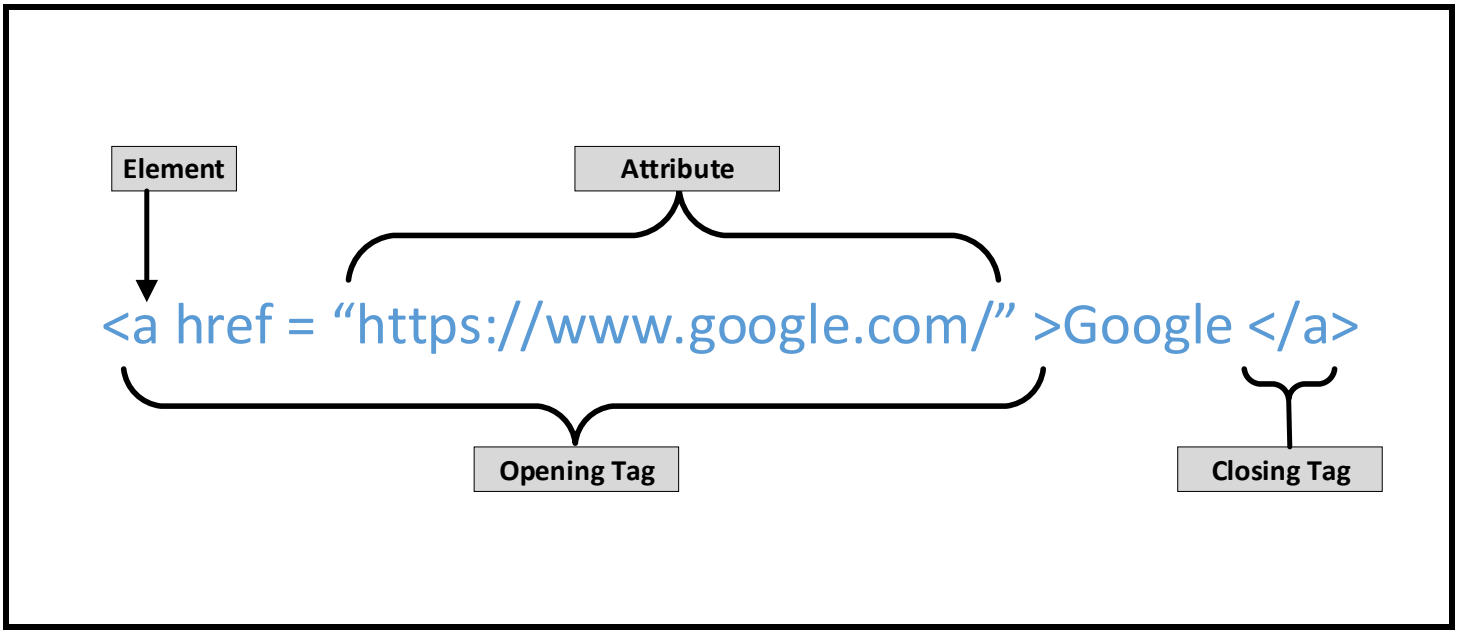


Figure 1: HTML Tag Syntax

2.3 HTML Document Structure

HTML documents are plain text documents saved with an .html file extension rather than a .txt file extension. To begin writing HTML, you first need a plain text editor that you are comfortable using. Sadly, this does not include Microsoft Word or Pages, as those are rich text editors. Two of the more popular plain text editors for writing HTML and CSS are Notepad++ (Windows) and Sublime Text (Max).

All HTML documents have a required structure that includes the following declaration and elements:

- <!DOCTYPE html>
- <html>
- <head>
- <body>

The document type declaration, or <!DOCTYPE html>, informs web browsers which version of HTML is being used and is placed at the very beginning of the HTML document. Because we'll be using the latest version of HTML, our document type declaration is simply <!DOCTYPE html>. Following the document type declaration, the <html> element signifies the beginning of the document.

Inside the <html> element, the <head> element identifies the top of the document, including any metadata (accompanying information about the page). The content inside the <head> element is not displayed on the web page itself. Instead, it may include the document title (which is displayed on the title bar in the browser window), links to any external files, or any other beneficial metadata.

All of the visible content within the web page will fall within the <body> element. A breakdown of a typical HTML document structure looks like this:

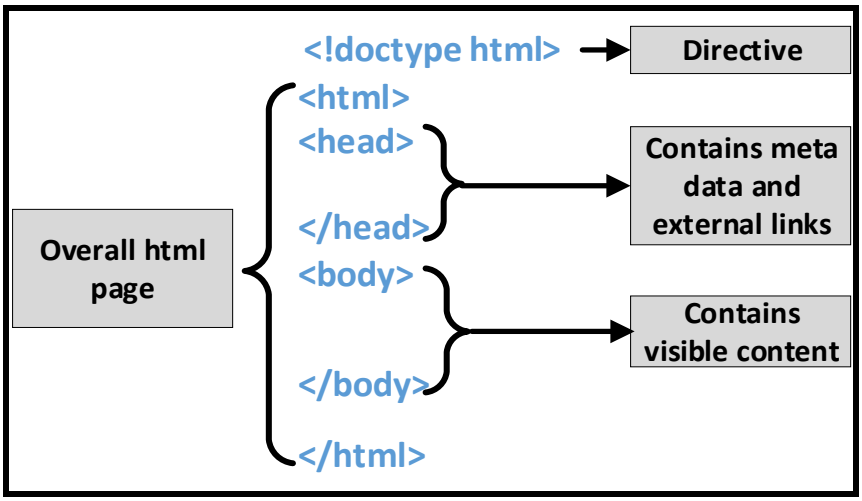


Figure 2: HTML Document Structure

A sample of a simple html page is shown below:

Code Example:

```
<!DOCTYPE html>
<html lang="en">
	<head>
		<meta charset="utf-8">
		<title>Hello World</title>
	</head>
	<body>
		<h1>Hello World</h1>
		<p>This is a web page.</p>
	</body>
</html>
```

The preceding code shows the document beginning with the document type declaration, <!DOCTYPE html>, followed directly by the <html> element. Inside the <html> element come the <head> and <body> elements.

The <head> element includes the character encoding of the page via the <meta charset="utf-8"> tag and the title of the document via the <title> element.

The <body> element includes a heading via the <h1> element and a paragraph via the <p> element. Because both the heading and paragraph are nested within the <body> element, they are visible on the web page.

When an element is placed inside of another element, also known as nested, it is a good idea to indent that element to keep the document structure well organized and legible. In the previous code, both the <head> and <body> elements were nested—and indented—inside the <html> element. The pattern of indenting for elements continues as new elements are added inside the <head> and <body> elements.

In the previous example, the <meta> element had only one tag and did not include a closing tag. This was intentional! Not all elements consist of opening and closing tags. Some elements simply receive their content or behavior from attributes within a single tag. The <meta> element is one of these elements. The content of the previous <meta> element is assigned with the use of the charset attribute and value. Other common self-closing elements include:

 <embed> <hr> <img> <input> <link> <meta> <param> <source> <wbr>

 **Note:** Self-closing elements are also called void elements. They also used to be written with an ending slash, such as <hr />

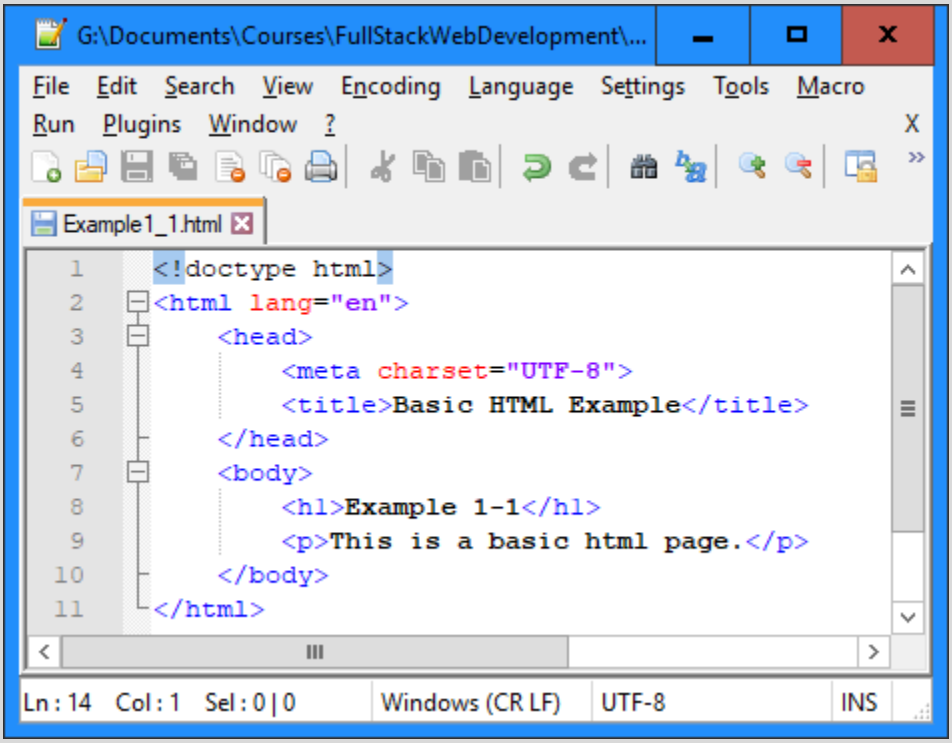
Code Validation

No matter how careful we are when writing our code, we will inevitably make mistakes. Thankfully, when writing HTML and CSS we have validators to check our work. The W3C has built both HTML (http://validator.w3.org/) and CSS (http://jigsaw.w3.org/css-validator/) validators that will scan code for mistakes.

Validating our code not only helps it render properly across all browsers, but also helps teach us the best practices for writing code.

Example 1-1: Basic HTML Page

1. Create a file named Example1_1.html containing the following code:



2. Save the file and execute it in a browser.